

Fleet Diagnostics Analytics Dashboard & DiagAI Assistant – Capability Guide

This guide details the capabilities of the Diagnostics Analytics Dashboard and the DiagAI assistant for engineering and executive audiences. It covers the full range of tools – from basic signal analysis to advanced AI-driven querying – with usage examples, best practices, and a summary of access methods for each tool.

1. Signal Analysis Tools

The dashboard provides comprehensive **signal analysis** utilities. Users can **select** any recorded signal (e.g. engine RPM, sensor voltages) for examination. Signals can be **plotted** as time-series charts (time on X-axis, signal value on Y-axis). Standard **statistics** (mean, max, min, RMS, etc.) are computed automatically, and users can visualize **distributions** via histograms to see the shape of the data's distribution. Signals may be **filtered** (e.g. low-pass or band-pass) to remove noise, improving clarity before further analysis. Cross-**correlation** or scatter plots can reveal relationships between two signals; the Pearson **correlation coefficient (r)** quantifies the linear association. Frequency-domain analysis is supported via **FFT**: transforming time signals into frequency spectra[\[1\]](#), which helps identify periodic faults. Users can also **compare** signals from multiple runs side-by-side to spot drift or regressions.

- **Best Practices:** Always inspect the raw time-series first. Apply noise filters (or windowing) to improve FFT accuracy. Use histograms to check for outliers or unexpected shapes in the data distribution. When analyzing multiple signals, align their time axes and consider standardizing units.

2. Diagnostic Analysis Tools

Built-in diagnostic analyses target common engine and system faults. **Misfire detection** automatically flags cylinders with abnormal combustion (signaled by vibration or OBD counters), since misfires degrade performance and increase fuel consumption and emissions. The system integrates OBD data: it decodes **DTCs (Diagnostic Trouble Codes)** and any proprietary **DFC (Fault Codes)** to explain fault conditions. It also tracks monitor performance: the **In-Use Performance Ratio (IUPR)** indicates how often an OBD monitor has actually run ($IUPR_m = \text{Numerator/Denominator for each monitor}$)[\[2\]](#). Specialized analyses include **gear hunting** detection (identifying vibrational patterns from meshing gears) and monitoring of **cylinder cut-out/skip-fire (CC/SL)** behavior for engines with variable displacement.

- **Best Practices:** When faults appear, correlate them with signal events. For example, confirm misfires by checking crankshaft vibration sensors or spark sensor logs. Review DTC/DFC histories in context: examine freeze-frame data and IUPR statistics^[2] to ensure monitors have completed. Verify gear-hunting signatures against known gear-mesh frequencies. For CC/SL analysis, ensure the timing of cylinder shut-offs is logged accurately.

3. Advanced Analytics

The dashboard can generate **empirical maps** and performance charts. For instance, it can build engine **BSFC (Brake-Specific Fuel Consumption)** and efficiency maps from logged data: plotting fuel consumption vs. speed and torque reveals the “sweet spots” of efficiency. An **iso-BSFC map** highlights the lowest fuel consumption regions (e.g. one example map shows a sweet spot at 206 g/kW·h, ~40.6% thermal efficiency). Users can analyze overall **engine performance** (power, torque, BSFC over test conditions) and create efficiency contour plots. Transmission insights are provided via **gear usage and transition statistics**: tracking how often each gear is used and shift timing to optimize shifts.

- **Usage Example:** Automatically generate a BSFC contour map by selecting engine speed and load signals; the tool will interpolate efficiency lines. Compare engine maps before/after tuning to quantify gains.
- **Best Practices:** Use stable, steady-state data points to build maps (avoid transient spikes). Validate maps against known benchmarks (e.g. dynamometer data). When analyzing gear usage, filter out incomplete cycles and normalize for run duration.

4. Visualization Features

The platform offers rich **interactive visualization** options. Plots are dynamic: users can **zoom**, pan, and hover for tooltips. Multiple signals can be plotted on shared or stacked axes for comparison. Scatter plots (points representing two signal values) reveal relationships; with interactivity, one can zoom into clusters or hover for details. **Heatmaps** display matrices of values as colors, ideal for showing data density or 2D distributions. For example, a BSFC map can be shown as a heatmap over speed vs. torque. 3D charts (surface or point clouds) visualize three variables together (e.g. performance surfaces).

Example of an interactive gauge/ dashboard visualization (a speedometer-style display)

for quick insight into key metrics. Interactive features allow drilling down into data: clicking a point in one view can highlight the related datapoint in another. Heatmaps provide a “bird’s-eye” view of large datasets, while scatter and 3D plots help identify clusters or outliers.

- **Best Practices:** Use clear axes labels and legends for all charts. For scatter plots, consider adding regression lines or contour lines to highlight trends. Select appropriate color scales in heatmaps to avoid misinterpretation (e.g. perceptually uniform palettes). Enable interactive annotations so users can track signal IDs or timestamps on hover.

5. Report Generation

The dashboard includes comprehensive **reporting**. Users can produce various report types (e.g. summary, detailed diagnostics, regulatory compliance) by selecting sections of analysis. Reports are **automatically generated**: the tool can assemble charts, tables, and text into a document. Sections and layout can be customized (choose which plots or metrics to include). Reports can be exported directly to **PDF** or image formats for sharing. For example, one can generate a weekly engine health report that compiles misfire logs, efficiency maps, and key signal statistics.

- **Best Practices:** Standardize report templates for consistency across tests. Use automation (scheduling or script triggers) to refresh reports after new data ingestion. Include executive summaries with key findings and embed visualizations for clarity. Since dashboards can export entire views as PDF, leverage this to capture complex visualizations without manual copying.

6. Automation & Integration

The system is designed for **automation** and easy integration into workflows. A RESTful **API** (or similar programmatic interface) allows engineers to query data, launch analyses, or pull results from external scripts. **Batch processing** is supported: users can run analysis jobs on multiple data files (e.g. overnight processing of test logs). Data can be **exported** in common formats (CSV, JSON, MAT, etc.) for use in other tools. The dashboard can integrate with data pipelines (e.g. via webhooks or scheduled tasks) so that new test data automatically triggers analysis.

- **Best Practices:** Automate routine analyses by writing scripts that call the API (e.g. to generate a report after each test cycle). Use standard data formats for interoperability. Maintain version-controlled analysis scripts and document the API endpoints. Ensure proper authentication and data access control for

integrations.

7. DiagAI Assistant

The DiagAI Assistant provides an **AI-driven natural-language interface** to the analytics system. Users can ask questions in plain English (or other supported languages) and receive intelligent answers. For example, an engineer might type “*What caused the cylinder 3 misfire on run 5?*” or “*Show torque vs. RPM for the last test*”. Advanced NLP interprets these queries and automatically selects the right analysis. As one NLP-powered tool notes, users can “ask sophisticated analytical questions using business terminology and get accurate results without needing to understand SQL or APIs”[\[3\]](#). The assistant supports context-aware dialog: it remembers prior questions, resolves pronouns, and can refine answers iteratively.

Behind the scenes, DiagAI uses **multi-agent orchestration**: specialized AI agents handle different tasks (e.g. signal querying, fault diagnosis, or plotting) and an orchestrator coordinates them. As IBM describes, “AI agent orchestration functions like a digital symphony. Each agent has a unique role and ... an orchestrator ... manages and coordinates their interactions. The orchestrator helps synchronize these specialized agents, ensuring that the right agent is activated at the right time for each task”. In practice, if a user’s query involves multiple domains (e.g. combining sensor analysis with DTC lookup), DiagAI routes sub-questions to the appropriate agents and then synthesizes the answer.

Usage Example: Possible user queries include:

- “List all active DTCs and their descriptions.”
- “Compare baseline vs. current fuel maps.”
- “Is engine knock above threshold at any point on run #12?”
- “Plot engine temperature and RPM during last 10 minutes.”

Each of these will prompt DiagAI to run the relevant analyses (e.g. filtering temperature signal, computing statistics, generating a chart) and return results with explanations.

- **Best Practices:** Phrase queries clearly using domain terms (e.g. “misfire,” “engine hours,” “lean spike”). If unsure, ask step-by-step (DiagAI can prompt for clarification or refine context). For multi-faceted queries, break them into simpler sub-questions. Review the sources or charts returned to validate DiagAI’s interpretation.

All advanced commands supported by the chat interface

Complete list of advanced chat commands

Basic operations

1. ``list_signals`` - List/search available signals
2. ``get_trips`` - Get all trips with metadata
3. ``stats`` - Calculate statistics (mean, min, max, std, percentiles) with time range support

Advanced statistics & analysis

4. ``compare_signals`` - Compare multiple signals side-by-side
5. ``correlation`` - Calculate Pearson/Spearman correlation between two signals
6. ``correlation_matrix`` - Correlation matrix with heatmap for multiple signals
7. ``distribution`` - Analyze signal distribution (histogram + box plot)
8. ``outliers`` - Detect outliers (IQR or Z-score method)
9. ``statistical_test`` - Statistical tests (t-test, Mann-Whitney, Kolmogorov-Smirnov)

Visualization

10. ``plot`` - Plot single signal (line, scatter, step) with time range support
11. ``plot_multiple`` - Plot multiple signals together (with subplots)
12. ``scatter_plot`` - Scatter plot between two signals with regression
13. ``heatmap`` - Create 3D heatmap for three signals
14. ``pie_chart`` - Pie chart for categorical/binning data
15. ``density_plot`` - KDE density plot showing probability distribution

Signal processing

16. ``rate_of_change`` - Calculate derivative/rate of change
17. ``integrate`` - Calculate integral/cumulative sum

- 18. ``fft_analysis`` - FFT frequency domain analysis
- 19. ``filter_signal`` - Apply filters (lowpass, highpass, bandpass)
- 20. ``smooth_signal`` - Smooth signal (moving average, exponential)
- 21. ``resample_signal`` - Resample to uniform timebase
- 22. ``advanced_signal_processing`` - Advanced processing (filter, FFT, denoise, peaks, resample)

Event detection

- 23. ``detect_events`` - Detect peaks/valleys in signal
- 24. ``threshold_violations`` - Detect threshold violations
- 25. ``detect_multi_condition_events`` - Complex multi-signal event detection

Vehicle-specific analysis

- 26. ``power_calculation`` - Calculate engine power from RPM and torque
- 27. ``efficiency_metrics`` - Calculate efficiency metrics (fuel consumption)
- 28. ``analyze_map`` - Analyze Engine Map (RPM vs Torque) with BSFC
- 29. ``analyze_bsfc`` - Analyze Brake Specific Fuel Consumption map
- 30. ``analyze_gear_usage`` - Analyze gear usage distribution (pie/bar chart)
- 31. ``analyze_by_gear`` - Analyze signal when in specific gear
- 32. ``analyze_signal_across_gears`` - Analyze signal across all gears
- 33. ``detect_gear_changes`` - Detect gear changes with optional conditions
- 34. ``calculate_acceleration_time`` - Calculate 0-100 km/h acceleration time

Fleet & trip analysis

- 35. ``trip_summary`` - Generate comprehensive trip summary
- 36. ``analyze_fleet_distance`` - Analyze fleet distance traveled
- 37. ``comprehensive_analysis`` - Comprehensive multi-signal analysis
- 38. ``generate_insights`` - LLM-powered insights from multiple signals

Data filtering & querying

- 39. ``filter_by_condition`` - Filter signal by condition (e.g., 'rpm<2000')
- 40. ``filter_by_condition_complex`` - Complex multi-signal filtering with AND/OR operators

Specialized diagnostic agents

- 41. `analyze\_with\_misfire\_agent` - Advanced misfire detection (5 algorithms)
- 42. `analyze\_with\_dfc\_agent` - DTC/DFC analysis with severity classification
- 43. `analyze\_with\_iupr\_agent` - IUPR monitoring and OBD-II compliance
- 44. `analyze\_with\_gear\_agent` - Gear hunting and transmission analysis
- 45. `analyze\_with\_fuel\_agent` - Fuel consumption and BSFC analysis
- 46. `analyze\_with\_wltp\_agent` - WLTP cycle analysis and emissions testing
- 47. `analyze\_with\_orchestrator` - Intelligent multi-agent coordination
- 48. `comprehensive\_diagnostic\_analysis` - Run ALL agents for complete vehicle health check

Advanced calibration & optimization

- 49. `drive\_cycle\_analysis` - RDE drive cycle generation and analysis
- 50. `two\_stage\_model\_calibration` - Two-stage model calibration for engine maps
- 51. `transmission\_optimization` - Optimize transmission shift schedules
- 52. `braking\_analysis` - Braking test analysis (distance, deceleration, ABS)
- 53. `dynamometer\_analysis` - Engine dynamometer analysis (torque/power curves)

Data quality & validation

- 54. `signal\_quality\_metrics` - Calculate signal quality (completeness, SNR, sample rate)

Curve fitting & modeling

- 55. `curve\_fit\_signal` - Fit curves (polynomial, exponential, logarithmic)

Report generation

- 56. `get\_report\_section` - Get diagnostic report sections (IUPR, DFC, Misfire, Fuel)

Export functions

- 57. `export` - Export data (CSV, Excel formats)
- 58. `export\_to\_pdf` - Export analysis to PDF report
- 59. `export\_to\_ppt` - Export analysis to PowerPoint presentation

Example natural language queries

Basic queries:

- "List all RPM signals"
- "Show statistics for torque signal"
- "Plot RPM from 60 to 120 seconds"
- "Compare RPM and torque signals"

Advanced analysis:

- "Detect misfires in the engine"
- "Analyze all DTCs and show severity"
- "Check IUPR compliance"
- "Is there gear hunting behavior?"
- "Calculate fuel efficiency map"
- "Run WLTP cycle analysis"

Complex queries:

- "Find all gear changes from 3rd to 4th when RPM > 4000"
- "Show torque in 4th gear only"
- "Analyze RPM across all gears"
- "Calculate 0-100 km/h acceleration time"
- "Filter data where rpm<2000 & torque>150"

Fleet analysis:

- "Generate comprehensive diagnostic report"
- "Analyze fleet distance traveled"
- "Compare fuel consumption across trips"
- "Find all vehicles with misfire issues"

Export & reports:

- "Export this analysis to PDF"
- "Generate PowerPoint presentation"
- "Get IUPR report section"

Total: 59 advanced commands

All commands support:

- Time range filtering (t0, t1 parameters)
- Trip ID filtering
- Automatic signal name matching (case-insensitive, fuzzy)
- Interactive Plotly visualizations
- Natural language interface via LLM

The LLM automatically selects the right command(s) based on your query, so you can ask in plain English.

This guide should help both engineers and executives understand the capabilities and best practices of the Diagnostics Analytics Dashboard and DiagAI assistant. Each section above includes examples and recommendations to facilitate practical use.

Summary of Tools & Access Methods

Category	Example Tools/ Features	Access Methods
Signal Analysis	Signal Selector panel, Time-series charts, FFT analyzer, Correlation matrix, Histogram tool	Interactive UI (dashboards), Data API, Script/CLI
Diagnostic Analysis	Misfire Detector, DTC/ DFC Browser, IUPR Monitor, Gear Hunting Analysis, CC/SL Checker	UI menus/forms, Automated alerts, Database queries
Advanced Analytics	Map Generator (BSFC, efficiency), Performance Summary, Gear Usage Stats	UI widgets (map builders), API endpoints
Visualization	Interactive Chart widgets, Multi-signal overlays, Scatter plots, Heatmaps, 3D plots	GUI/drag-and-drop; Export to SVG/PNG; Custom scripting
Reporting	PDF Report Builder, Template Editor, Scheduled Reports	Web UI (report wizard), API trigger (automated jobs)
Automation & Integration	RESTful Data API, Batch Job Scheduler, Data Export (CSV/ JSON)	Programmatic (scripts/ CLI), CI/CD pipelines, Plug-ins
DiagAI Assistant	Natural Language Query Interface, Contextual Chat, Multi- agent Diagnostics	Chat/voice UI, Chatbot APIs (Slack/Teams integrations)